

Reviewer Pack — Minimal Number of Automorphism Generators and Circuit Monoto...

Entry 2c2b520aef31 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 13:12:24 UTC

Minimal Number of Automorphism Generators and Circuit Monotone Width Inequality

Entry ID: 2c2b520aef31 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 13:12:24 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Automorphic Forms Theory) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every d -regular Boolean circuit C with n inputs, the minimal number of generators required for an automorphism group action on its gates is upper bounded by the square root of the circuit monotone width $w_m(C)$, i.e., $|G| \leq \sqrt{w_m(C)}$, where G is the automorphism group of C .

Rationale (proposer's reasoning):

Automorphic forms in algebraic number theory have a rich connection to symmetry and group actions. This conjecture proposes that the structure of symmetry, as captured by the automorphism group, is related to the complexity of circuits through monotone width, which is a measure of circuit complexity.

Taxonomy category: AutomorphicForms-BooleanCircuitComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d2bbd7fcaec6628a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient (r) between the minimal number of automorphism generators $|G|$ and the square root of the monotone width $\sqrt{w_m(C)}$ is ≥ 0.7 for at least 80% of the 30 seeds, and the p-value from a significance test with $\alpha = 0.05$ indicates no significant difference from zero correlation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_circuit(n, d):
        if (d * n) % 2 != 0:
            return None
        circuit = [[random.choice([0, 1]) for _ in range(d)] for _ in range(n)]
        return circuit

    def compute_monotone_width(circuit):
        # Simplified monotone width calculation (for demonstration)
        return len(circuit) * len(circuit[0])

    def is_automorphism(gate1, gate2, circuit):
        if len(gate1) != len(gate2):
            return False
        for i in range(len(gate1)):
            if gate1[i] != gate2[(i + 1) % len(gate1)]:
                return False
        return True

    def compute_automorphism_group(circuit):
        n = len(circuit)
        d = len(circuit[0])
        automorphisms = []
        for perm in itertools.permutations(range(n)):
            if all(is_automorphism(circuit[i], circuit[perm[i]], circuit) for i in range(n)):
                automorphisms.append(perm)
        return automorphisms

    def compute_min_generators(automorphisms):
        generators = set()
        for a in automorphisms:
            if len(a) == 1:
```

```

        generators.add(a[0])
    return len(generators)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    circuit = generate_d_regular_circuit(n, 2)
    if circuit is None:
        continue
    w_m = compute_monotone_width(circuit)
    G = compute_automorphism_group(circuit)
    if not G:
        continue
    |G| = compute_min_generators(G)
    results.append({
        "n": n,
        "|G|": |G|,
        " $\sqrt{w_m(C)}$ ": math.sqrt(w_m),
    })

metric_name = "min_automorphism_generators"
metric_value = sum(result["|G|"] for result in results) / len(results)
instances_tested = len(results)
n_max = max(result["n"] for result in results)
conjecture_holds = all(math.sqrt(result["w_m(C)"]) >= result["|G|"] for result in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample,
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete its analysis and verify the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within a reasonable timeframe.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15358
2	propose	ollama_remote	glm4:latest	0	0	19223
3	preregistration	ollama_remote	glm4:latest	0	0	9962
4	novelty	ollama_remote	glm4:latest	0	0	8230
5	novelty	ollama_remote	glm4:latest	0	0	8723
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20329
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13915
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13574
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11956
10	judge	ollama_remote	glm4:latest	0	0	12038

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 133308 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/2c2b520aef31.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2c2b520aef31.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2c2b520aef31.tar.gz (if generated)